Database II Lab/ 3rd Grade

[Second Semester] and [2026]

[Lab 8] [7 /4/ 2026] [8.30 + 10.30 am]

Instructor Information

Instructor

Dr. Rasool Hisham

Email Hours

Dr. Zainab Namh

Dr. Azhar Flaih

[rasool.hisham@nahrainuniv.edu.iq] [2 Hrs]

Assist. Prof. Zahraa Jaaz

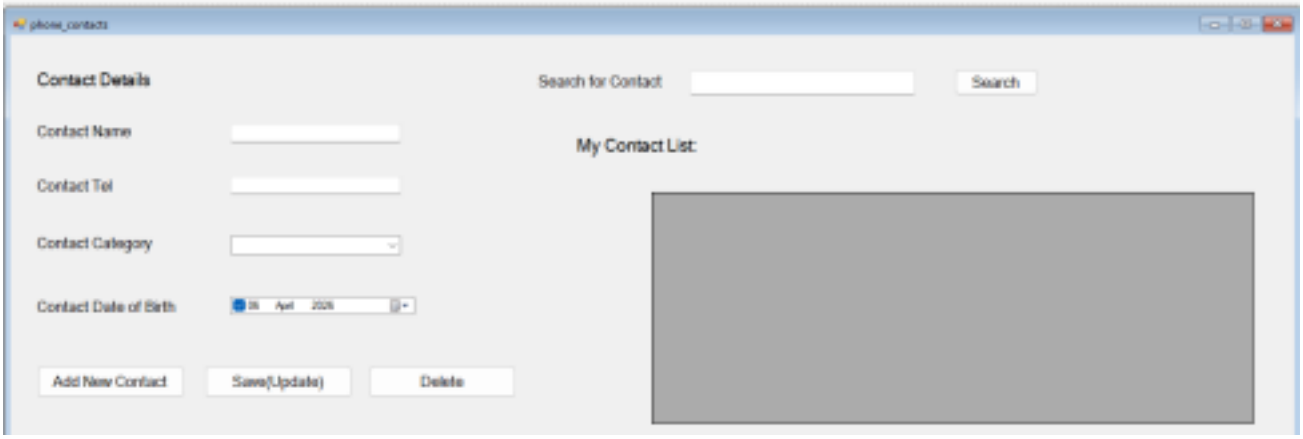
CRUD Operations with DataTable and DataAdapters

Lab Objectives

By the end of this lab, the student should be able to:

- Understand the role of DataTable as an in-memory data store.
- Distinguish between MySqlDataReader and MySqlDataAdapter.
- Display data using a DataGridView.
- Use nullable types in C# (DateTime?) for flexible data entry.
- Perform form validation before database operations.
- Handle date controls for date entry.
- Perform full CRUD operations on the phone_book database:
  - Create — Add new contact
  - Read — View contact data in a grid
  - Update — Modify an existing contact record
  - Delete — Remove a contact from the database

Part 1 - Main Interface Form



Add a DataGridView (GrdContacts) and DateTimePicker (DtpBirthday)

Part 2— Core Concepts Before Coding

What is a DataTable?

A DataTable is an in-memory representation of data stored as rows and columns — like a database table that lives inside your program's memory. It belongs to the System.Data namespace.

- Holds data temporarily before it is displayed or saved.
- Works as the 'middleman' between the database and the DataGridView.
- Can be filled either by a DataAdapter or DataReader.

What is a DataAdapter?

MySqlDataAdapter is a .NET class that serves as a **bridge between a MySQL database and a DataTable**, automatically executing a SQL command, reading the results, and filling the table — handling the connection open/close cycle internally.

What is a DataGridView?

A DataGridView is a powerful Windows Forms control used to display tabular data in a grid format — similar to a spreadsheet. Key characteristics:

- Displays data as rows and columns.
- Binds to various data sources, most commonly a DataTable.
- Used when you need to display and optionally let the user choose records.

How They Work Together

You often use a **DataTable** to store your data and then **bind it** to a **DataGridView** to display it on the screen.

C# — DataGridView Binding Concept

```
DataTable table = new DataTable();
dataGridView.DataSource = table;
```

Part 3 — MySqlDataReader vs. MySqlDataAdapter

Both are used to retrieve data from a database, but they work in fundamentally different ways.

Aspect	MySqlDataReader (+ Load)	MySqlDataAdapter (+ Fill)

How the data table is filled?	Manually: open connection → execute reader → table.Load(reader) → close	Automatically: adapter.Fill(table) Adapter will open and close the connection.
Flexibility without a DataTable	Can be used directly to populate ListBox/ComboBox without any DataTable	Always produces a DataTable (or DataSet)

MySqlDataReader — The Manual Approach

With MySqlDataReader, a connection must be opened, then a command is executed, and read row by row. The connection must remain open while reading:

```
// Using MySqlDataReader to fill a DataTable manually


using (var conn = new MySqlConnection(connectionString))
 {
  conn.Open();

  string query = "SELECT ContactId, ContactName, ContactTel,
CategoryName FROM contacts c LEFT JOIN contact_category cc
ON  c.ContactCategoryID = cc.ContactCategoryID";

  using (var cmd = new MySqlCommand(query, conn))
  {
  using (MySqlDataReader reader = cmd.ExecuteReader())  {

  DataTable dt = new DataTable();
  dt.Load(reader); // Load data from reader into DataTable
GridContacts.DataSource = dt; //Bind DataTable to DataGridView  }
  }
 }
```

MySqlDataAdapter — The Automatic Approach

With MySqlDataAdapter, the adapter manages the connection lifecycle for you. It opens, reads all data into the DataTable, then closes the connection automatically:

Full LoadData() with Line-by-Line Annotations

```
private void loadadapter()
 {

  // LINE 1: Create an empty in-memory table
```

```
 DataTable table = new DataTable();

 // LINE 2: Create connection object (not yet open)
 using (var conn = new MySqlConnection(connectionString))
{
 // LINE 3: Open the physical connection to MySQL
 conn.Open();

 // LINE 4: Write the SQL query as a string
 string sql = "SELECT ContactId, ContactName, ContactTel,
CategoryName FROM contacts c LEFT JOIN contact_category cc ON
c.ContactCategoryID = cc.ContactCategoryID";

 // LINE 5: Attach the SQL query to the connection
```



```
using (MySqlCommand cmd = new MySqlCommand(sql, conn))  {
 // LINE 6: Create adapter
 var adapter = new MySqlDataAdapter(cmd);

 // LINE 7: Execute / Fill

 adapter.Fill(table);

 } // Variables 'cmd' and 'adapter' are cleaned up here  }
 GrdContacts.DataSource = table; //Bind DataTable to DataGridView }
```

The 'using' block guarantees the connection is closed even if an error occurs. Always wrap connections in 'using' statements.

Notice: adapter.Fill(table) on LINE 7 is where the magic happens. The adapter internally opens the connection (already open), runs the SQL, converts each result row into a DataRow, adds it to the DataTable, then returns.

Here is the precise logical sequence from button click to data appearing on screen:

Step	Action	What Happens
1	Form loads	Form_Load event fires
2	LoadData() is called	Method body begins executing
3	DataTable table = new DataTable()	Empty table created in RAM
4	new MySqlConnection(connectionString)	Connection object created (but not open)
5	conn.Open()	Physical TCP connection opened to MySQL server

6	MySqlCommand(sql, conn)	SQL text + connection packaged into a command
7	new MySqlDataAdapter(cmd)	Adapter created — it knows the command to run
8	adapter.Fill(table)	Adapter executes SQL, maps rows → DataTable rows
9	using block ends	conn.Close() called automatically
10	GridContacts.DataSource = table	DataGridView reads table rows and renders the grid

Part 4 - DateTime Picker (DtpBirthday)

A DateTimePicker has a built-in dropdown calendar

- Stores the value as a real DateTime property (DtpBirthday.Value), not a string
- DtpBirthday.Checked is the **DateTimePicker's own built-in checkbox** (enabled by setting ***ShowCheckBox = true*** in properties).

```
// Using the built-in checkbox of the Picker
// Reading the value —already a DateTime Type

DateTime? dob = DtpBirthday.Checked ?
DtpBirthday.Value.Date :(DateTime?)null
```

To save the value into the database

```
cmd.Parameters.AddWithValue("@dob", DtpBirthday.Checked
?  DtpBirthday.Value.Date: (object)DBNull.Value);
```

The cast to (object) is required because the ternary operator needs both sides to be the same type — DBNull.Value and DateTime are different types, so casting one side to object satisfies the compiler.

Or if you prefer it more readable across two lines:

```
if (DtpBirthday.Checked)
 cmd.Parameters.AddWithValue("@dob", DBNull.Value);
else
 cmd.Parameters.AddWithValue("@dob", DtpBirthday.Value.Date);
```

Both do exactly the same thing. The .**Date** at the end strips the time portion (sets it to 00:00:00) so only the date is saved to the MySQL DATE column — without it you'd be sending a full DateTime with a time component which MySQL would still accept but it's cleaner to strip it.

Part 5 - BtnDelete_Click — Delete Selected Row (Line by Line)

This functionality allows the user to delete an existing contact from the database.

Steps:

1. When the "Delete" button (BtnDel) is clicked, the BtnDel_Click event is triggered.
2. The event checks if a row is selected in the DataGridView. If a row is selected, it retrieves the ContactId of the contact to be deleted.

3. A DELETE SQL query is executed on the MySQL database, removing the selected contact record.
4. After the deletion is completed, the loadadapter () method is called again to refresh the grid and remove the deleted contact from the display.

```
private void BtnDelete_Click(object sender, EventArgs
e) {
try{
 // Step 1: Check that the user has actually selected a row
if (GridContacts.SelectedRows.Count > 0)
 {

 // Step 2: Access the first selected row
 DataGridViewRow row = GridContacts.SelectedRows[0]; // or var

 // Step 3: Read the ContactId from the selected row
var contactId = row.Cells["ContactId"].Value;

 // Step 4: Execute the DELETE query

 …



 // Step 5: Refresh the grid
 loadadapter ();
}
else
{
 MessageBox.Show("Please select a contact to delete.");
}
} catch …
```

GridContacts.SelectedRows	This property returns a COLLECTION of all currently selected rows in the DataGridView
SelectedRows	A Collection of DataGridViewRow objects.
GridContacts.SelectedRows.Count	The number of rows currently selected (0 if none)
GridContacts.SelectedRows[0]	Accesses the first row in the selected row collection. If at least one row is selected, it retrieves that row as a DataGridViewRow object.
row.Cells["ContactId"]	The cell in the ContactId column of that row
row.Cells["ContactId"].Value	The actual value stored in that cell

Part 6 - GrdContacts_SelectionChanged — Fill the controls with Selected Data

This is the event that fires automatically every time the user clicks a different row in GrdContacts. Its job is to read the selected row's data and fill all the form controls (TextBoxes, DateTimePicker, ComboBox) so the user can review and edit them.

Steps:

The event checks if a row is selected in the DataGridView. If a row is selected, it retrieves the selected contact details and the selected details are filled to the controls.

The Logic — Step by Step

Step	Code	What Happens
1	GrdContacts.CurrentRow.Index	Get the currently row from the grid
2	row.Cells["ContactName"].Value?.ToString()	Fill TxtFullName and TxtPhone from the row's cells
3	row.Cells["ContactBirthday"].Value != DBNull.Value	Check if birthday is null before assigning to DateTimePicker
4	row.Cells["ContactCategoryID"].Value	Read the hidden category ID (integer) from the row
5	foreach (var item in CboCateg.Items)	Loop through ComboBox items to find matching category
6	((dynamic)item).Value == targetId	Compare each item's hidden Value to targetId, select when matched

```
private void GrdContacts_SelectionChanged(object sender, EventArgs
e)  // fires on every row click

if (GrdContacts.CurrentRow.Index >= 0)
{

 DataGridViewRow row = GrdContacts.CurrentRow;


 TxtFullName.Text = row.Cells["ContactName"].Value?.ToString();
TxtPhone.Text = row.Cells["ContactTel"].Value?.ToString();

 string contactId = row.Cells["ContactId"].Value.ToString();

 if (row.Cells["ContactBirthday"].Value != DBNull.Value)
{
 DtpBirthday.Value =
Convert.ToDateTime(row.Cells["ContactBirthday"].Value);
 DtpBirthday.Checked = true;
 }
 else
 {
 // If there IS NO date, reset it to today and uncheck the box
```



```
 DtpBirthday.Value = DateTime.Now;
 DtpBirthday.Checked = false; // This requires ShowCheckBox = true  in
properties
 }

 var categoryIdValue = row.Cells["ContactCategoryID"].Value;


 if (categoryIdValue == DBNull.Value)
 {

 CboCateg.SelectedIndex = -1;
 }
 else
 {
 int targetId = Convert.ToInt32(categoryIdValue);


 foreach (var item in CboCateg.Items)
 {
 if (((dynamic)item).Value == targetId)
 {
 CboCateg.SelectedItem = item;
 break;
 }
 }
```

Part 7 - BtnUpdateGrid_Click (Update a Contact)

After the user clicks a row (controls fill automatically), they can modify TxtFullName, TxtPhone, DtpBirthday, and CboCateg, then click the Update button to save the changes back to the database.

The Logic — Step by Step

Step	Code	What Happens
1	GrdContacts.SelectedRows.Count > 0	Confirm a row is selected
2	row.Cells["ContactId"].Value	Read the ContactId — this is the WHERE clause value
3	TxtFullName.Text, TxtPhone.Text	Read updated name and phone from the TextBoxes
4	DtpBirthday.Checked	If checked → use DtpBirthday.Value.Date; else → DBNull.Value
5	CboCateg.SelectedItem	If selected → read its .Value (int ID); else → DBNull.Value
6	UPDATE contacts SET ... WHERE ContactId = @id	Execute the parameterized UPDATE query
7	loadadapter()	Refresh the grid to show the updated data

Both @dob and @categ are declared as type object. This is required because AddWithValue needs a single type, and we might pass either a real value (DateTime / int) or DBNull.Value. Declaring as object accommodates both cases.

NULL Handling Reference

Use this table as a quick reference when working with nullable database columns in C#:

Read a nullable string	reader["col"] == DBNull.Value ? "default" : reader.GetString("col")
Read a nullable int	reader["col"] == DBNull.Value ? -1 : reader.GetInt32("col")
Read a nullable DateTime	reader["col"] == DBNull.Value ? (DateTime?)null : reader.GetDateTime("col")
Write NULL to database	cmd.Parameters.AddWithValue("@p", value == -1 ? (object)DBNull.Value : value)
Check before GetDateTime	Never call GetDateTime() on a NULL column — always check DBNull.Value first

What is DateTime?

In C#, DateTime? is a **nullable** type for the DateTime structure. The ? after DateTime indicates that the variable can either hold a valid DateTime value or be **null**. Because DOB might be null in the database — this avoids conversion errors.

What is DBNull.Value?

It is a special value in .NET that represents a **database NULL** value, commonly encountered when dealing with databases.

Understanding DateTime? and DBNull.Value

DateTime?	A NULLABLE DateTime. The '?' means the variable can hold a valid date OR null. Used because birthday might be missing in the database.
DBNull.Value	A special .NET value representing a database NULL. Returned when a cell/column contains no value (NULL in SQL).
Why not just use null?	In .NET, a non-nullable DateTime cannot be null. DateTime? (DateTime-nullable) allows null. Without '?', assigning null would cause a compile error.
The ternary pattern	row.Cells["col"].Value == DBNull.Value ? (DateTime?)null : (DateTime)row.Cells["col"].Value — check first, then cast.